

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 07

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

Nº de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 14/10/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para el primer ejercicio retomé el rover con jerarquía que agregué en la práctica anterior, agregando los modelos correspondientes a las seis llantas que faltaban ya separadas de las patas y cargándolos en OpenGL mediante la clase Model.

```
Model llantafrontalmasz;
Model llantafrontalmenosz;
Model llantamediomasz;
Model llantamediomenosz;
Model llantatraseramasz;
Model llantatraseramenosz;
```

```
llantafrontalmasz = Model();
llantafrontalmasz.LoadModel("Models/llantafrontal+z.obj");
llantafrontalmenosz = Model();
llantafrontalmenosz.LoadModel("Models/llantafrontal-z.obj");
llantamediomasz = Model();
llantamediomasz.LoadModel("Models/llantamedio+z.obj");
llantamediomenosz = Model();
llantamediomenosz.LoadModel("Models/llantamedio-z.obj");
llantatraseramasz = Model();
llantatraseramasz.LoadModel("Models/llantatrasera+z.obj");
llantatraseramenosz = Model();
llantatraseramenosz.LoadModel("Models/llantatrasera-z.obj");
```

Posteriormente agregué las rotaciones de las articulaciones del brazo y la pinza, utilizando las articulaciones 1, 2 y 3 respectivamente. De la misma manera agregué las rotaciones de las articulaciones 4 a 9 para las ruedas del rover, conservando la jerarquía del modelo y utilizando glm::rotate para controlar sus movimientos.

También agregué las transformaciones de las seis llantas, haciendo que giren al avanzar o retroceder el rover mediante las articulaciones 10 y 11.

```
//Rover con jerarquia
model = matrizRover;
modelaux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
roverbase.RenderModel();
//Base Brazo
model = modelaux;
model = glm::translate(model, glm::vec3(-2.5f, 1.5f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
brazobase.RenderModel();
//Tubo 1
model = glm::translate(model, glm::vec3(0.8f, 1.1f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
brazouno.RenderModel();
//Tubo 2
model = glm::translate(model, glm::vec3(2.5f, 2.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
brazodos.RenderModel();
//Pinza
model = glm::translate(model, glm::vec3(-2.5f, 3.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
pinza.RenderModel();
//Llanta frontal +Z
model = modelaux;
model = glm::translate(model, glm::vec3(-2.2f, 1.2f, 3.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
frontalmasz.RenderModel();
model = glm::translate(model, glm::vec3(-3.6f, -2.7f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantafrontalmasz.RenderModel();
```

```
//Llanta frontal -Z
model = modelaux;
model = glm::translate(model, glm::vec3(-2.2f, 1.2f, -3.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
frontalmenosz.RenderModel();
model = glm::translate(model, glm::vec3(-3.6f, -2.7f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantafrontalmenosz.RenderModel();
//Llanta media +Z
model = modelaux;
model = glm::translate(model, glm::vec3(2.2f, 0.5f, 3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
mediomasz.RenderModel();
model = glm::translate(model, glm::vec3(-1.5f, -2.1f, 1.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantamediomasz.RenderModel();
//Llanta media -Z
model = modelaux;
model = glm::translate(model, glm::vec3(2.0f, 0.5f, -3.1f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
mediomenosz.RenderModel();
model = glm::translate(model, glm::vec3(-1.5f, -2.1f, -1.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantamediomenosz.RenderModel();
```

```
//Llanta trasera +Z
model = modelaux;
model = glm::translate(model, glm::vec3(4.0f, 0.5f, 3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
traseramasz.RenderModel();
model = glm::translate(model, glm::vec3(1.5f, -2.1f, 1.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantatraseramasz.RenderModel();
//Llanta trasera -Z
model = modelaux;
model = glm::translate(model, glm::vec3(4.2f, 0.5f, -3.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion9()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
traseramenosz.RenderModel();
model = glm::translate(model, glm::vec3(1.5f, -2.1f, -1.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
llantatraseramenosz.RenderModel();
```

Después implementé el movimiento del rover hacia adelante y atrás, utilizando las articulaciones 10 y 11 para calcular el desplazamiento sobre el eje X. Para esto creé una matriz llamada matrizRover, que contiene las transformaciones del

modelo y que posteriormente utilizo para dibujar el rover junto con todas sus articulaciones.

```
// Movimiento Rover
GLfloat avanceRover = 0.05f * (mainWindow.getarticulacion10() - mainWindow.getarticulacion11());
glm::mat4 matrizRover = glm::mat4(1.0f);
matrizRover = glm::translate(matrizRover, glm::vec3(1.0f - avanceRover, 0.0f, 6.0f));
matrizRover = glm::scale(matrizRover, glm::vec3(0.4f, 0.4f, 0.4f));
matrizRover = glm::translate(matrizRover, glm::vec3(0.0f, 1.0f, 0.0f));
```

Para agregar la luz frontal declaré una nueva luz de tipo spotlight en la posición 2 del arreglo spotLights, configurando su color, intensidad, dirección y valores de atenuación.

Posteriormente calculé la posición y dirección del faro a partir de matrizRover, utilizando glm::vec4 para transformar sus coordenadas locales y glm::normalize para obtener la dirección de iluminación. Finalmente utilicé SetFlash para actualizar la luz dentro del ciclo while, permitiendo que siga al rover cuando avanza o retrocede.

```
// Luz frente Rover
glm::vec3 posicionFaroRover = glm::vec3(matrizRover * glm::vec4(-5.0f, 1.0f, 0.0f, 1.0f));
glm::vec3 direccionFaroRover = glm::normalize(glm::vec3(matrizRover * glm::vec4(-1.0f, -0.5f, 0.0f, 0.0f)));
spotLights[2].SetFlash(posicionFaroRover, direccionFaroRover);
```

Para el segundo ejercicio agregué el movimiento del avión, utilizando las articulaciones 12 y 13 para subir y bajar, y las articulaciones 14 y 15 para avanzar y retroceder. Para esto creé matrizAvion, en la que se aplican las transformaciones de traslación, escala y rotación del modelo.

También utilicé una matriz de rotación de -90 grados sobre el eje Y para calcular correctamente la dirección hacia adelante del avión, de acuerdo con la orientación que ya tenía.

```
// Movimiento del Avion
GLfloat alturaAvion = 0.05f * (mainWindow.getarticulacion12() - mainWindow.getarticulacion13());
GLfloat avanceAvion = 0.05f * (mainWindow.getarticulacion14() - mainWindow.getarticulacion15());
glm::mat4 giroAvion = glm::rotate(glm::mat4(1.0f), -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glm::vec3 frenteLocalAvion = glm::vec3(0.0f, 0.0f, -1.0f);
glm::vec3 frenteAvion = glm::normalize(glm::vec3(giroAvion * glm::vec4(frenteLocalAvion, 0.0f)));
glm::mat4 matrizAvion = glm::mat4(1.0f);
matrizAvion = glm::translate(matrizAvion, glm::vec3(0.0f, 5.0f + alturaAvion, 6.0f) + frenteAvion * avanceAvion);
matrizAvion = glm::scale(matrizAvion, glm::vec3(0.3f, 0.3f, 0.3f));
matrizAvion = glm::rotate(matrizAvion, -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
```

Posteriormente declaré una segunda luz spotlight en la posición 3 del arreglo, colocándola debajo del avión y estableciendo su dirección hacia el eje Y negativo. Para que la luz siguiera el movimiento del modelo, calculé su posición y dirección utilizando matrizAvion y actualicé sus valores mediante SetFlash.

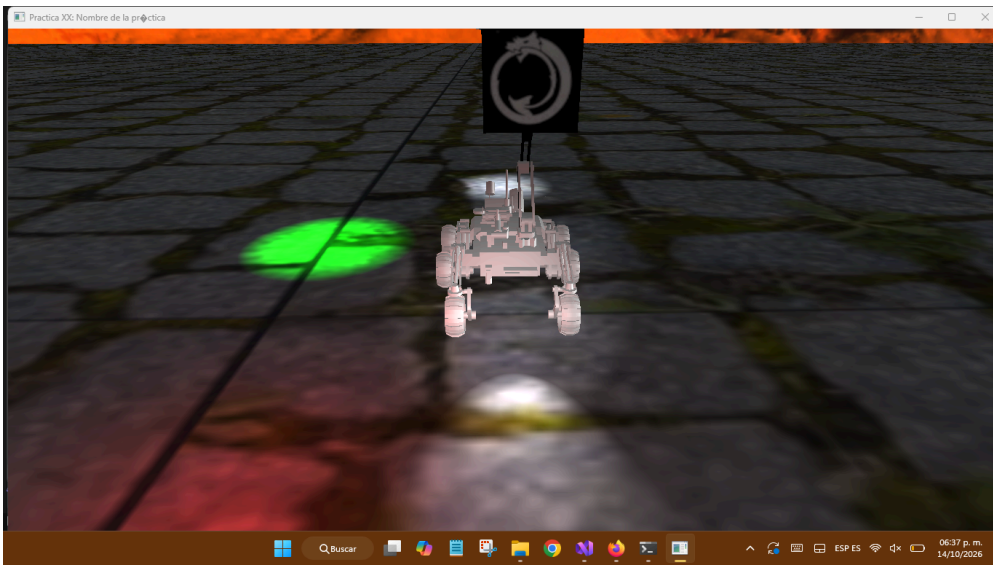
Para el último ejercicio agregué otra luz spotlight al avión, esta vez en la parte frontal, utilizando la posición 4 del arreglo. De igual manera calculé su posición y dirección utilizando matrizAvion, pero estableciendo la dirección local hacia el eje Z negativo para que iluminara hacia adelante después de aplicar la rotación del modelo.

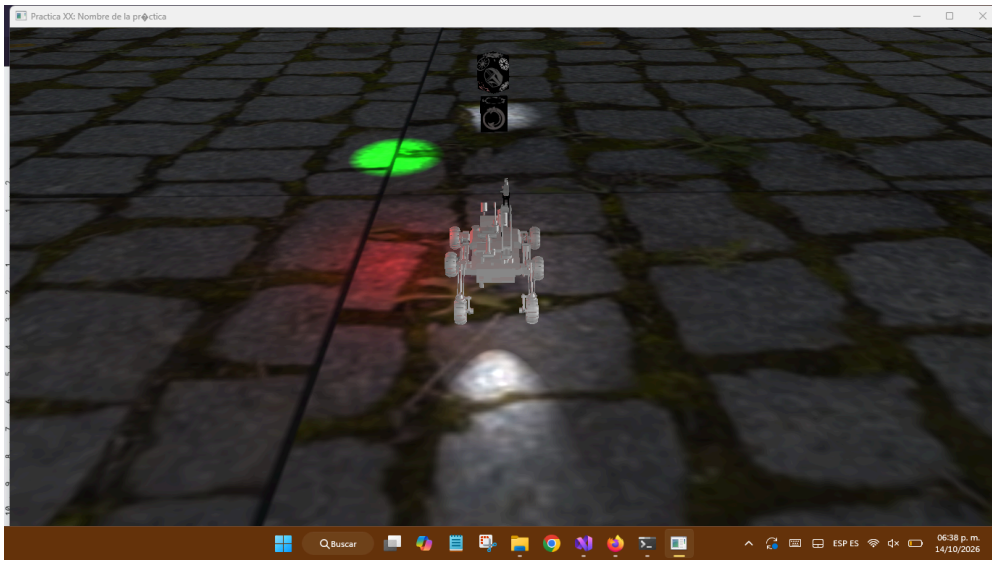
```
// Luz bajo Avion
glm::vec3 posicionFaroInferior = glm::vec3(matrizAvion * glm::vec4(0.0f, -2.0f, 0.0f, 1.0f));
glm::vec3 direccionFaroInferior = glm::normalize(glm::vec3(matrizAvion * glm::vec4(0.0f, -1.0f, 0.0f, 0.0f)));
spotLights[3].SetFlash(posicionFaroInferior, direccionFaroInferior);
// Luz frente Avion
glm::vec3 posicionFaroFrontal = glm::vec3(matrizAvion * glm::vec4(0.0f, 0.0f, -3.0f, 1.0f));
glm::vec3 direccionFaroFrontal = glm::normalize(glm::vec3(matrizAvion * glm::vec4(0.0f, 0.0f, -1.0f, 0.0f)));
spotLights[4].SetFlash(posicionFaroFrontal, direccionFaroFrontal);
```

Finalmente modifiqué los valores de atenuación de la linterna que sigue a la cámara, agregando valores de 0.01 para las componentes lineal y cuadrática, ya que anteriormente ambos valores eran cero y la intensidad de la luz no disminuía con la distancia.

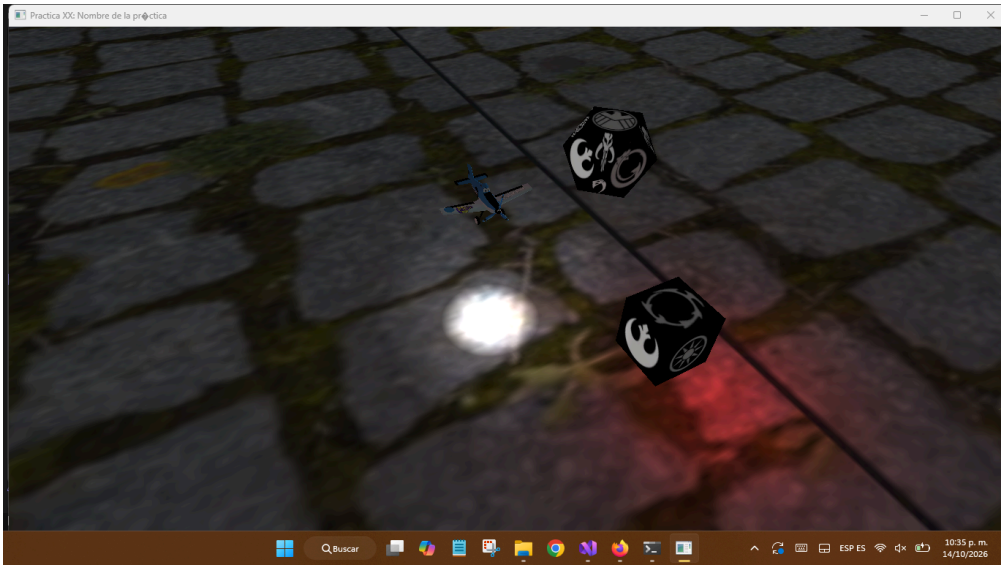
```
unsigned int spotLightCount = 0;
//linterna
spotLights[0] = SpotLight(1.0f, 1.0f, 1.0f,
    0.3f, 1.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, -1.0f, 0.0f,
    1.0f, 0.01f, 0.01f,
    15.0f);
spotLightCount++;
```

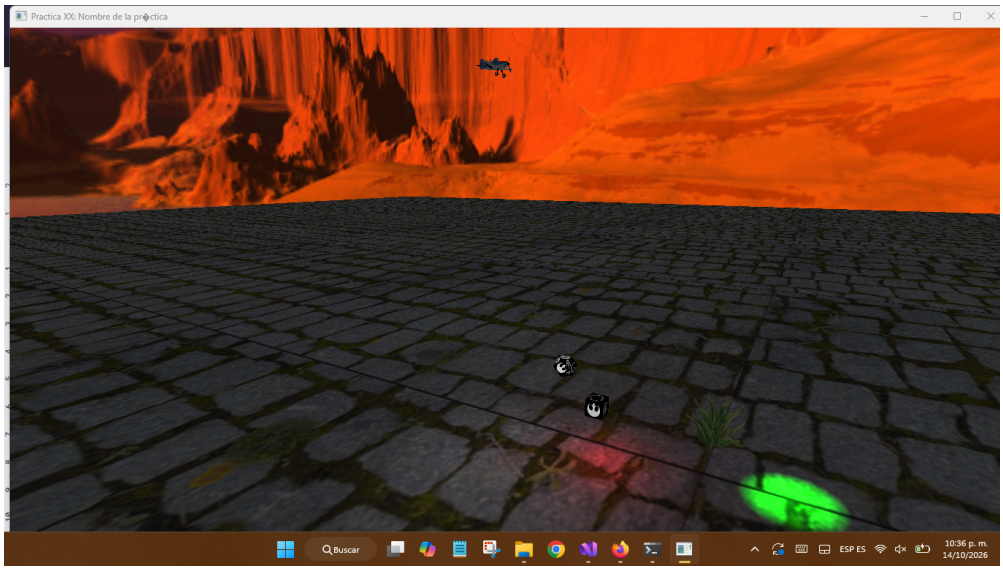
La ejecución final se muestra a continuación:

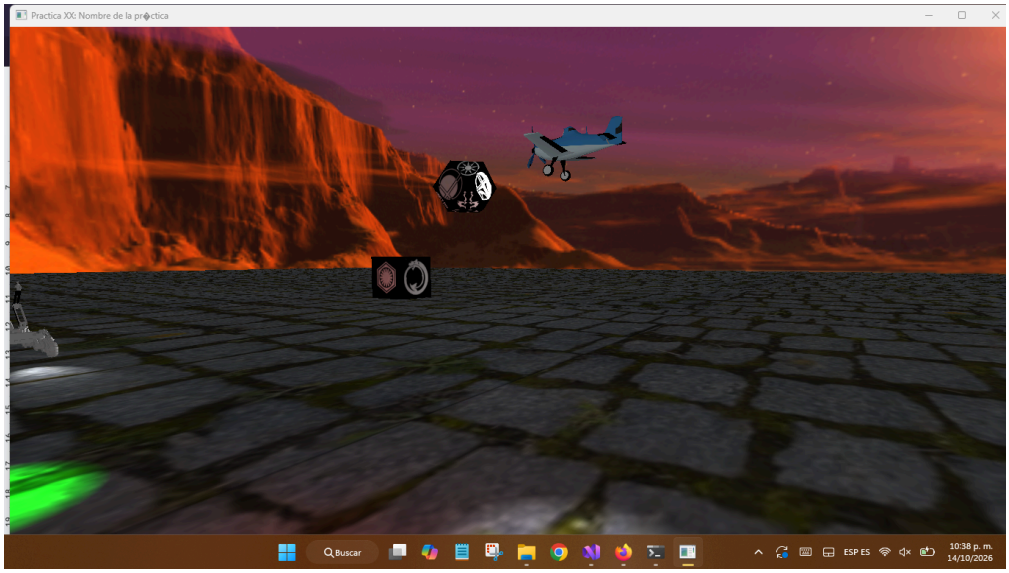



Practica XX: Nombre de la pr◆ctica
Buscar
ESP ES
06:38 p. m.
14/10/2026


Practica XX: Nombre de la pr◆ctica
Buscar
ESP ES
06:39 p. m.
14/10/2026


Practica XX: Nombre de la pr◆ctica
Buscar
ESP ES
10:35 p. m.
14/10/2026


Practica XX: Nombre de la pr@ctica
Buscar
ESP ES
10:36 p. m.
14/10/2026


Practica XX: Nombre de la pr@ctica
Buscar
ESP ES
10:38 p. m.
14/10/2026


Practica XX: Nombre de la pr@ctica
Buscar
ESP ES
10:38 p. m.
14/10/2026



2. Problemas presentados.

No tuve ningún problema en la solución de la práctica.

3. Conclusión:

Con la realización de esta práctica pude comprender mejor cómo implementar luces de tipo spotlight utilizando transformaciones jerárquicas para que se mantengan unidas a los modelos tridimensionales. El uso de matrices permitió relacionar la posición y dirección de las luces con los movimientos del rover y del avión, sin modificar la jerarquía que ya tenía implementada. También comprendí mejor la importancia de la orientación de los modelos y de los parámetros de atenuación para obtener una iluminación más adecuada. Aunque fue necesario realizar ajustes en las direcciones de movimiento e iluminación, la práctica me ayudó a entender mejor el manejo de luces dinámicas en OpenGL.